x86-64 Assembly Language Workshop Guide

Complete Beginner Guide — myCompiler.io

PART 1: WHAT IS ASSEMBLY LANGUAGE?

Assembly language is the lowest level programming language that humans can read. It talks directly to the CPU using simple instructions.

```
Python / C++  →  High Level Language
Assembly      →  Middle Level (human readable CPU instructions)
Machine Code  →  Binary 0s and 1s (what CPU actually runs)
```

When you write assembly, the assembler converts it to machine code and the CPU executes it directly.

PART 2: HOW myCompiler.io WORKS

```
Step 1    Go to mycompiler.io
Step 2    Select New Project
Step 3    Choose ASM x86-64
Step 4    Type your code in main.asm file
Step 5    Click the green Run button
Step 6    See output on the right side panel
```

Important rules on myCompiler:

```
Use double quotes " for strings
Every instruction on its own line
Never use pipe symbol | between instructions
File must be named main.asm
```

PART 3: THREE THINGS YOU MUST KNOW

1. Sections

```
section .data    →   where you store your variables and strings
section .text    →   where you write your actual code
```

2. Registers

Registers are tiny storage boxes inside the CPU itself:

```
Register    Size      Use
--------    ----      ---
RAX         64 bit    Main math register and syscall number
RBX         64 bit    Second number for math
RCX         64 bit    Counter for loops
RDX         64 bit    Remainder after division
```

```
RDI          64 bit    syscall argument 1
RSI          64 bit    syscall argument 2
AL           8 bit     Lower part of RAX, store one byte
DL           8 bit     Lower part of RDX, store remainder byte
```

3. Syscalls

Syscall means asking the operating system to do something:

```
RAX = 1    →   write (print something to screen) cout
RAX = 60   →   exit (end the program) {return 0}
```

To print, load these four registers then call syscall:

```
RAX = 1          syscall number for write int a = 1
RDI = 1          1 means stdout (the screen)output on the screen
RSI = message    what you want to print cout <<"HELLO WORLD"
RDX = length     how many bytes to print
```

To exit, load these two registers then call syscall:

```
RAX = 60         syscall number for exit loop (exit;)
RDI = 0          exit code 0 means success (either completed or not)
```

PART 4: THE ASCII TRICK

The CPU only understands numbers. To print a digit on screen you must add 48 to it first. This converts the number to its ASCII character code.

Your Number	Add 48	ASCII Code	Prints On Screen
0	+ 48	48	0
1	+ 48	49	1
2	+ 48	50	2
3	+ 48	51	3
4	+ 48	52	4
5	+ 48	53	5
6	+ 48	54	6
7	+ 48	55	7
8	+ 48	56	8
9	+ 48	57	9

So always write this before printing a digit:

```
add rax, 48
```

Keep all math results between 0 and 9 for single digit printing.

PART 5: ALL PROGRAMS

Program 1 — Hello World

```
section .data
    msg db "Hello World", 10
    len equ $ - msg

section .text
global _start
_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, len
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
Hello World
```

Explanation:

```
msg db "Hello World", 10    defines the string, 10 is newline character
len equ $ - msg             automatically calculates length of string
global _start               marks the entry point of the program
mov rax, 1                  syscall number for write
mov rdi, 1                  stdout
mov rsi, msg                what to print
mov rdx, len                length of message
syscall                     execute the print
mov rax, 60                 syscall number for exit
mov rdi, 0                  exit code success
syscall                     execute the exit
```

Program 2 — Move Values Between Registers

```
section .text
global _start
_start:
    mov rax, 10
    mov rbx, rax
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
no visible output
```

Explanation:

```
mov rax, 10     puts number 10 into RAX
mov rbx, rax    copies value from RAX into RBX
This teaches how mov works between registers
```

Program 3 — ADDITION (5 + 3 = 8)

```
section .data
    result db 0, 10

section .text
global _start
_start:
    mov rax, 5
    add rax, 3
    add rax, 48
    mov [result], al
    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 2
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
8
```

Explanation:

```
mov rax, 5        load first number into RAX
add rax, 3        RAX = RAX + 3 = 8
add rax, 48       convert 8 to ASCII code 56 which prints as 8
mov [result], al  store the byte into result variable
then print and exit
```

Program 4 — SUBTRACTION (9 - 4 = 5)

```
section .data
    result db 0, 10

section .text
global _start
_start:
    mov rax, 9
    sub rax, 4
    add rax, 48
    mov [result], al
    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 2
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
5
```

Explanation:

```
mov rax, 9    load 9 into RAX
sub rax, 4    RAX = RAX - 4 = 5
add rax, 48   convert 5 to ASCII
then print and exit
```

Program 5 — MULTIPLICATION (3 x 3 = 9)

```
section .data
    result db 0, 10

section .text
global _start
_start:
    mov rax, 3
    mov rbx, 3
    mul rbx
    add rax, 48
    mov [result], al
    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 2
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
9
```

Explanation:

```
mov rax, 3    first number goes into RAX
mov rbx, 3    second number goes into RBX
mul rbx       RAX = RAX x RBX = 9
              mul always multiplies with RAX automatically
              result is always stored back into RAX
add rax, 48   convert to ASCII then print and exit
```

Program 6 — DIVISION (8 / 2 = 4)

```
section .data
    result db 0, 10

section .text
global _start
_start:
    mov rax, 8
    mov rbx, 2
```

```
    xor rdx, rdx
    div rbx
    add rax, 48
    mov [result], al
    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 2
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
4
```

Explanation:

```
mov rax, 8      dividend goes into RAX
mov rbx, 2      divisor goes into RBX
xor rdx, rdx    clears RDX to zero, always required before div
div rbx         RAX = RAX / RBX = 4   and   RDX = remainder
add rax, 48     convert quotient to ASCII then print and exit
```

Program 7 — MODULO / Remainder (7 mod 3 = 1)

```
section .data
    result db 0, 10

section .text
global _start
_start:
    mov rax, 7
    mov rbx, 3
    xor rdx, rdx
    div rbx
    add rdx, 48
    mov [result], dl
    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 2
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
1
```

Explanation:

```
div rbx         after division RAX = quotient   RDX = remainder
add rdx, 48     we add 48 to RDX not RAX because we want remainder
```

```
mov [result], dl   use dl not al because remainder is in RDX
```

Program 8 — Print Label With Result (4 + 5 = 9)

```
section .data
    msg db "Sum=", 0
    result db 0, 10

section .text
global _start
_start:
    mov rax, 4
    add rax, 5
    add rax, 48
    mov [result], al
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, 4
    syscall
    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 2
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:

```
Sum=9
```

Explanation:

```
first syscall block prints the label Sum=
second syscall block prints the digit 9
two separate print calls combined give Sum=9
```

PART 6: PROGRAM SKELETON — COPY EVERY TIME

Use this as your starting template for every new program:

```
section .data
    result db 0, 10

section .text
global _start
_start:

    ; WRITE YOUR MATH HERE
    ; example:
    ; mov rax, 5
    ; add rax, 3
```

```
    add rax, 48
    mov [result], al

    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 2
    syscall

    mov rax, 60
    mov rdi, 0
    syscall
```

Only change the math section. Everything else stays exactly the same every time.

PART 7: INSTRUCTIONS QUICK REFERENCE

Instruction	Example	Meaning
mov	mov rax, 5	put value 5 into RAX
mov	mov rbx, rax	copy RAX value into RBX
add	add rax, 3	RAX = RAX + 3
sub	sub rax, 4	RAX = RAX - 4
mul	mul rbx	RAX = RAX x RBX
div	div rbx	RAX = RAX / RBX RDX = remainder
xor	xor rdx, rdx	set RDX to zero
syscall	syscall	execute OS call

PART 8: DIVISION RULES — VERY IMPORTANT

Rule 1	Dividend always goes into RAX before dividing
Rule 2	Divisor goes into RBX
Rule 3	Always clear RDX with xor rdx,rdx before dividing
Rule 4	After div: RAX = quotient RDX = remainder
Rule 5	To print quotient use add rax,48 and mov [result],al
Rule 6	To print remainder use add rdx,48 and mov [result],dl

PART 9: COMMON ERRORS AND FIXES

Error Message	Cause	Fix
unterminated string missing backtick	Used backtick or wrong quote	Use double quotes only
label expected at start of line	Pipe symbol used	Put each instruction on new line
segmentation fault	Wrong syscall number	RAX=1 for print RAX=60 for exit
undefined symbol _start	Missing global line	Write global _start before _start:
wrong number printed on screen	Forgot to add 48	Always add rax,48 before printing
program crashes during division	RDX was not cleared	Write xor rdx,rdx before div
remainder not printing correctly	Used al instead of dl	Use dl for remainder dl is part of RDX

```
nothing prints on screen              Wrong RSI register            RSI must
point to your result variable
```

PART 10: GOLDEN RULES

```
Rule 1     Every instruction must be on its own separate line
Rule 2     Never use pipe symbol | between instructions
Rule 3     Always write global _start above _start:
Rule 4     Always end every program with syscall 60 or it will crash
Rule 5     Always write xor rdx,rdx before any division
Rule 6     mul always multiplies with RAX no need to mention RAX
Rule 7     Always add 48 to digit before printing it on screen
Rule 8     Use al not rax when writing mov [result],al
Rule 9     Use dl not rdx when storing remainder after division
Rule 10    Keep all math results between 0 and 9 for single digit output
Rule 11    Use double quotes for strings never backticks or single quotes
Rule 12    section .data comes before section .text always
```

PART 11: WORKSHOP EXERCISE ORDER

```
Exercise 1    Type Hello World exactly as shown and click Run
Exercise 2    Change Hello World to your own name
Exercise 3    Run Addition program with 5+3, change to 4+4 and run again
Exercise 4    Run Subtraction program with 9-4, change to 8-2 and run again
Exercise 5    Run Multiplication with 3x3, change to 2x4 and run again
Exercise 6    Run Division with 8/2, change to 9/3 and run again
Exercise 7    Run Modulo with 7 mod 3, try 8 mod 3 and predict answer first
Exercise 8    Run Label program, change Sum= to Ans= and run again
Challenge 1   Write a program that calculates 6+3 and prints Ans=9
Challenge 2   Write a program that calculates 8-5 and prints Result=3
Challenge 3   Write a program that calculates 4x2 and prints Mul=8
```